

TALLER: SISTEMA INTELIGENTE DE MONITOREO Y SEGURIDAD PARA INVERNADEROS

Tamara Valeria Escobar Andrade, Santiago Rodríguez Bermeo y Thomas Trujillo Cerquera

Facultad de Ingeniería, Programa Ingeniería de Sistemas

Corporación Universitaria del Huila – CORHUILA

Microcontroladores – Grupo I

Álvaro Hernán Alarcón López

Abril, 2026

Tabla de contenido

Introducción	6
Introduction	7
Objetivos	8
Objetivo General	8
Objetivos Específicos	8
Marco Teórico	9
Microcontroladores y el ESP32	9
Sensores de Variables Ambientales	9
Sensor de Temperatura y Humedad DHT22	9
Fotoresistor (LDR)	10
Sensores de Distancia y Movimiento	11
Sensor Ultrasónico de Distancia HC-SR04	11
Actuadores Simulados (LEDs)	12
MicroPython y la Lógica del Sistema	12
Desarrollo	13
Herramientas y Elementos Utilizados	13
Herramientas:	13
Elementos simulados:	13
Análisis del Problema	13
Temperatura	14
Humedad	14
Luminosidad	14
Nivel de agua en tanque	14
Control manual de emergencia	15
Elección de componentes	15

Sensor DHT22	15
Sensor de luminosidad LDR	16
Sensor ultrasónico HC-SR04	16
Leds	16
Buzzer	16
Lógica de Control del Sistema	19
Validación del sistema	26
Control de temperatura	26
Control de humedad	28
Control de nivel del tanque	30
Control de iluminación	33
Resultados experimentales	35
Conclusión	37
Referencias	38

Lista de Tablas

Tabla 1. Rangos de Operación del Sistema	15
Tabla 2. Asignación de pines del ESP32	18
Tabla 3. Mediciones simuladas.....	36

Lista de figuras

Figura 1. Diagrama de pines (pinout) del DOIT ESP32 DevKit V1. Fuente: [3].	9
Figura 2. Módulo sensor de temperatura y humedad DHT22 con sus pines de conexión (Vcc, Datos, NC, GND). Fuente: [5].	10
Figura 3. Pinout y componentes del módulo sensor fotoresistor LDR. Fuente: [8].	11
figura 4. Módulo de sensor ultrasónico HC-SR04 con sus pines de conexión (Vcc, Trig, Echo, GND). Fuente: [9].	11
Figura 5. Conexiones del circuito	18
Figura 6. Definición de variables y librerías en el programa	20
Figura 7. Definición variables humedad y temperatura, además de función de interrupción	21
Figura 8. Función de alerta, lectura DHT22 y control de temperatura	22
Figura 9. Función de control de humedad y medición de distancia	23
Figura 10. Función control de distancia en llenado del tanque y cálculo de lux	24
Figura 11. Función de control de iluminación	25
Figura 12. Ciclo while	26
Figura 13. Activación del sistema ante temperatura baja.	27
Ilustración 14. Activación del sistema de ventilación por alta temperatura.	28
Figura 15. Activación del sistema de extracción por alta humedad y temperatura.	29
Figura 16. Activación del sistema de nebulización por baja humedad.	30
Figura 17. Estado de llenado del tanque de agua.	31
Figura 18. Detección de nivel bajo del tanque.	32
Figura 19. Detección de tanque lleno.	33
Figura 20. Ciclo de iluminación artificial por baja intensidad lumínica.	34
Figura 21. Activación del modo de emergencia del sistema	35

Introducción

El manejo de un invernadero implica mucho más que simplemente proteger los cultivos, ya que el estado de las plantas depende directamente de condiciones como la temperatura, la humedad, la cantidad de luz y la disponibilidad de agua. En la práctica, controlar estas variables de forma manual puede ser complicado, especialmente cuando se presentan cambios constantes en el ambiente, lo que puede afectar el crecimiento de los cultivos o generar un uso poco eficiente de los recursos.

Por esta razón, en este trabajo se plantea el desarrollo de un sistema de monitoreo y control que permita supervisar estas condiciones de manera más organizada y automática. Para ello, se utiliza el microcontrolador ESP32 junto con diferentes sensores que permiten obtener información del entorno en tiempo real, la cual es procesada para generar respuestas como alertas o activación de dispositivos.

A lo largo del informe se describe el proceso de diseño del sistema, la selección de los componentes utilizados y la lógica implementada para su funcionamiento. Además, se presentan pruebas realizadas en un entorno de simulación, con el fin de verificar que el sistema responde de manera adecuada ante distintas condiciones, mostrando así su posible aplicación en escenarios reales.

Introduction

Managing a greenhouse involves more than just protecting crops, since plant development depends heavily on factors such as temperature, humidity, light levels, and water availability. In real conditions, controlling these variables manually can be difficult, especially when environmental changes occur frequently, which may affect crop growth or lead to inefficient use of resources.

For this reason, this project focuses on the development of a monitoring and control system that allows these conditions to be supervised in a more organized and automated way. The system is based on the ESP32 microcontroller and integrates different sensors to collect real-time data from the environment, which is then used to trigger actions such as alerts or system responses.

Throughout this report, the design process of the system is described, including the selection of components and the control logic implemented. In addition, simulation tests are presented to verify that the system behaves as expected under different conditions, showing its potential application in realworld greenhouse environments.

Objetivos

Objetivo General

Diseñar e implementar un sistema inteligente de monitoreo y seguridad para un invernadero, basado en microcontroladores, que permita supervisar variables ambientales como temperatura, humedad, luminosidad y nivel de agua, generando alertas automáticas ante condiciones fuera de los rangos establecidos.

Objetivos Específicos

- Medir y controlar la temperatura del invernadero para mantenerla dentro de rangos óptimos mediante la activación de sistemas de ventilación o calefacción.
- Monitorear la humedad relativa del ambiente para evitar condiciones que afecten el desarrollo de las plantas, activando sistemas de humidificación o extracción cuando sea necesario.
- Evaluar la intensidad de la luz para garantizar niveles adecuados de iluminación, activando luces artificiales en condiciones de baja luminosidad.
- Detectar el nivel de agua en los tanques de riego mediante sensores de distancia, generando alertas ante niveles críticos de llenado o vaciado.
- Implementar un sistema de alertas automáticas que informe sobre condiciones anómalas en el invernadero.
- Incorporar un mecanismo de control manual que permita activar o desactivar el sistema de alertas en situaciones de emergencia.
- Desarrollar y simular el sistema en un entorno virtual (Wokwi), verificando el correcto funcionamiento de los sensores y la lógica programada en MicroPython.

Marco Teórico

Microcontroladores y el ESP32

Un microcontrolador es un circuito integrado programable que contiene una unidad central de procesamiento (CPU), memoria y periféricos de entrada/salida en un solo chip. Diseñados para ejecutar tareas específicas en sistemas embebidos, estos dispositivos permiten interactuar con sensores y actuadores para automatizar procesos [1].

Para este proyecto de invernadero, el núcleo del sistema es la tarjeta de desarrollo ESP32 DevKit V1. Como se detalla en su "pinout" (distribución de pines) en la **Figura 1**, este potente SoC (System on a Chip) destaca por su microcontrolador de doble núcleo Tensilica Xtensa LX6, conectividad Wi-Fi y Bluetooth integrada, y una amplia gama de pines GPIO (General Purpose Input Output) multifuncionales. Estas características lo hacen ideal para aplicaciones de Internet de las Cosas (IoT) y automatización agrícola que requieren monitoreo y control en tiempo real [2].

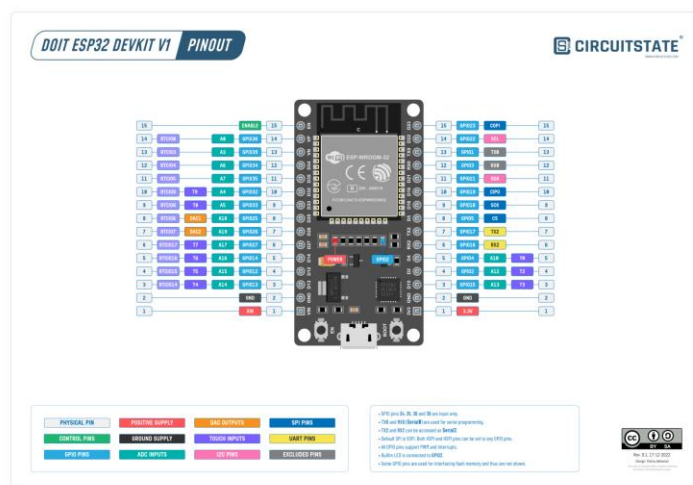


Figura 1. Diagrama de pines (pinout) del DOIT ESP32 DevKit V1. Fuente: [3].

Sensores de Variables Ambientales

Los sensores son dispositivos que detectan y responden a algún tipo de entrada del entorno físico, convirtiendo la magnitud física medida en una señal eléctrica analógica o digital que puede ser interpretada por un microcontrolador [4]. En un invernadero automatizado, los sensores monitorean las variables críticas que afectan el crecimiento de las plantas.

Sensor de Temperatura y Humedad DHT22

El DHT22 (también conocido como AM2302) es un sensor digital de temperatura y humedad que destaca por su mayor precisión y rango de medición en comparación con modelos anteriores como el DHT11 **Figura 2**. Utiliza un sensor de humedad capacitivo y un termistor para medir el aire circundante, enviando una señal digital directamente al ESP32 a través de un solo pin [5].



Figura 2. Módulo sensor de temperatura y humedad DHT22 con sus pines de conexión (Vcc, Datos, NC, GND). Fuente: [5].

En el invernadero, este sensor es fundamental para asegurar que las plantas crezcan en un ambiente óptimo, permitiendo activar sistemas de ventilación o riego si las condiciones se desvían de los rangos preestablecidos.

Fotoreistor (LDR)

Un fotoreistor o LDR es un componente electrónico cuya resistencia disminuye a medida que aumenta la intensidad de la luz incidente. Este dispositivo es fundamental en el invernadero para monitorear los niveles de iluminación natural tal como se muestra en la **Figura 3** [6]. Gracias a su comportamiento analógico, permite al ESP32 determinar si es necesario activar iluminación artificial suplementaria para optimizar la fotosíntesis de los cultivos [7].

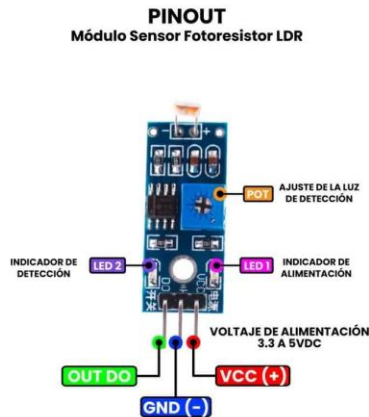


Figura 3. Pinout y componentes del módulo sensor fotoresistor LDR. Fuente: [8].

Sensores de Distancia y Movimiento

Sensor Ultrasónico de Distancia HC-SR04

El HC-SR04 es un sensor digital que mide la distancia a un objeto emitiendo ráfagas de ondas ultrasónicas (con una frecuencia de 40 kHz, inaudible para el oído humano) y calculando el tiempo que tarda la señal en rebotar y regresar [9]. Consta de un transmisor ultrasónico (Trigger) y un receptor (Echo) como se muestra en la **Figura 4**.



figura 4. Módulo de sensor ultrasónico HC-SR04 con sus pines de conexión (Vcc, Trig, Echo, GND). Fuente: [9].

El principio de funcionamiento se basa en la fórmula de la velocidad:

$$Distancia = \frac{(tiempo\ de\ viaje \cdot velocidad\ del\ sonido)}{2}$$

Donde la velocidad del sonido es aproximadamente 343 m/s a 20°C, y el tiempo de viaje es la duración del pulso en el pin Echo [9]. Para el invernadero automatizado, este sensor puede utilizarse para monitorear el nivel de agua en un tanque de reserva para riego. Midiendo la distancia desde la tapa del tanque hasta la superficie del agua, el sistema puede alertar sobre un bajo nivel de agua o desactivar la bomba de riego para protegerla de daños.

Actuadores Simulados (LEDs)

Un actuador es un componente que convierte una señal eléctrica en un movimiento mecánico, luz, calor u otra forma de energía física para realizar una acción [10]. En este proyecto, utilizamos Diodos Emisores de Luz (LEDs) para simular visualmente la activación de los actuadores reales del invernadero. Un LED es un dispositivo semiconductor que emite luz cuando una corriente eléctrica fluye a través de él en la dirección correcta (polarización directa) [11]. Para controlar un LED con el ESP32, se conecta a un pin GPIO configurado como salida digital.

- **HIGH (1/3.3V):** El pin entrega voltaje, permitiendo que la corriente fluya y encienda el LED.
- **LOW (0/0V):** El pin corta el voltaje, apagando el LED.

MicroPython y la Lógica del Sistema

Para programar la lógica del invernadero en el ESP32, utilizamos MicroPython, una implementación ligera y eficiente de Python 3 optimizada para microcontroladores [12]. Este lenguaje simplifica la interacción con el hardware, permitiendo configurar pines GPIO, leer valores analógicos y digitales, y controlar los LEDs actuadores de forma sencilla e intuitiva.

Desarrollo

Herramientas y Elementos Utilizados

Para el sistema de monitoreo y control del invernadero se utilizaron herramientas de simulación y programación, las cuales permitieron validar su comportamiento sin necesidad de tener el hardware y los elementos en físico. A continuación, se lista los recursos utilizados:

Herramientas:

- Plataforma de simulación wokwi
- Lenguaje de programación Mycropython
- Computador portátil

Elementos simulados:

- Sensor ultrasónico HC-SR04
- Sensor DHT22 de temperatura y humedad
- Sensor de luminosidad LDR
- 6 resistencias de 220 Ω
- 6 leds de diferentes colores
- 1 switch
- 1 buzzer
- Cables de conexión
- Microcontrolador ESP32

Análisis del Problema

En un invernadero es sumamente importante mantener controladas las condiciones ambientales para el crecimiento óptimo de las plantas. Debido a esto, es necesario manejar ciertas variables que influyen en la salud de la especie, entre estas encontramos la temperatura, la humedad, la luminosidad y el nivel de agua. A partir de las situaciones críticas de cada factor se diseña al sistema automatizado del invernadero capaz de generar acciones o alertas cuando se presente condiciones fuera de los rangos. En el caso de la simulación cada acción será representado por medio de un led diferente. Además, el sistema permite una opción muy útil en casos de emergencia, el cual es un control manual de desactivación u activación.

En este sentido, se definen las principales condiciones de operación del sistema:

Temperatura

Mantener a la planta dentro de un rango evita que presente daños por exceso de frío o calor. Los cambios bruscos de esta variable producen afectaciones en el crecimiento y la floración de las especies.

Se establece un rango de operación, en el cual, si la temperatura supera los 37 °C, el sistema activa mecanismos de ventiladores o la apertura de ventanas. Por otra parte, si la temperatura cae debajo de los 2°C, el sistema activa humidificación. Las acciones correctivas permanecen activas hasta que la variable regrese a un valor dentro del rango establecido.

Humedad

La humedad será relativa en el invernadero, es decir será representado a través de porcentaje. Esta es una de las variables más importantes a controlar, ya que si no se mantiene un manejo adecuado puede llevar a la proliferación de enfermedades fúngicas o estrés hídrico, lo cual afecta de manera prolongada la salud de la planta. En el sistema propuesto, se establecen condiciones de control basadas en rangos específicos. Si la humedad cae por debajo de 40%, se active un sistema de nebulización, pero si la humedad está por encima de 80 % y en el mismo momento su temperatura está por encima de 37 °C se activará un mecanismo de extractores.

Luminosidad

La cantidad de luz durante el día afecta el rendimiento de la fotosíntesis, en este caso vamos a evaluarlo para plantas de día largo, donde se estableció un fotoperiodo de 14 horas de luz (12 natural + 2 artificial), lo cual favorece el crecimiento vegetativo y, dependiendo de la especie, puede inducir o retrasar la floración. Este control del fotoperiodo es fundamental en invernaderos para regular el ciclo de vida de las plantas. El rango de operación es que, si la luz está por debajo de 100 lux, se enciende los leds para completar la iluminación.

Nivel de agua en tanque

Se hace uso de tanques de riego donde se controla el nivel de agua para evitar desbordamiento o falta de agua. Entre su rango de operación, se establece que si la distancia medida es mayor a 180 cm se activa una alerta de “Tanque vacío” o llenado de bomba. Sin embargo, si la distancia es baja, es decir menos a 20 cm, significa que el tanque está lleno y debe cerrar la entrada de las válvulas para evitar desbordamientos.

Control manual de emergencia

El sistema incorpora un mecanismo de control manual mediante un interruptor (switch), el cual permite activar o desactivar el sistema de alertas en caso de emergencia.

Esta funcionalidad proporciona al usuario la posibilidad de intervenir directamente en el funcionamiento del sistema, garantizando mayor seguridad y control ante situaciones imprevistas.

Para dejar más claro las condiciones de cada rango de operación, en la Tabla 1 se presentan los rangos de operación definidos para cada variable del sistema, así como las condiciones críticas y las acciones correspondientes.

Tabla 1. Rangos de Operación del Sistema

Variable	Rango Óptimo	Condición Crítica	Acción del Sistema
Temperatura	2 °C – 37 °C	> 37 °C	Activar ventilación (LED)
Temperatura	2 °C – 37 °C	< 2 °C	Activar humidificación
Humedad	40 % – 80 %	< 40 %	Activar nebulización
Humedad	40 % – 80 %	> 80 % y T > 37 °C	Activar extractores
Luminosidad	≥ 100 lux	< 100 lux	Encender iluminación LED
Nivel de agua	20 cm – 180 cm	> 180 cm	Alerta “tanque vacío” / activar llenado
Nivel de agua	20 cm – 180 cm	< 20 cm	Detener entrada de agua

Elección de componentes

Para el desarrollo del sistema se realizó la selección de componentes considerando su funcionalidad, compatibilidad con el microcontrolador ESP32 y facilidad de implementación en el entorno de simulación. Se eligieron sensores y actuadores capaces de medir y representar las variables ambientales previamente definidas, permitiendo una adecuada adquisición de datos y la ejecución de acciones dentro de la lógica del sistema. A continuación, se presentan los componentes seleccionados y las razones de su elección.

Sensor DHT22

Se escogió este sensor debido a su capacidad para medir tanto la temperatura como la humedad, variables que se miden en el invernadero, por lo que permite una operación más eficiente. Además, el sensor entrega datos digitales, lo que facilita la lectura a través del microcontrolador ESP32. Por otra parte, opera con un voltaje de alimentación entre **3.3 V y 5 V** y entrega datos en formato digital, lo que facilita su integración con el microcontrolador.

Sensor de luminosidad LDR

Se utilizó el sensor LDR debido a que permite determinar la cantidad de luz que se encuentra en el entorno. En este caso, se utilizó el módulo LDR, el cual ya incluye los componentes necesarios para su funcionamiento, evitando la necesidad de implementar resistencias adicionales en el circuito. Este módulo opera típicamente con un voltaje de 3.3 V o 5 V, y proporciona una señal analógica proporcional al nivel de iluminación.

Sensor ultrasónico HC-SR04

Se empleó este sensor ultrasónico para medir el nivel de agua en un tanque, cabe destacar que como este sensor presenta errores de medición al medir a distancias muy cercanas o lejanas, es perfecto para este caso ya que la distancia que maneja no sobrepasa sus límites de operación. Este dispositivo funciona con un voltaje de 5 V, por lo que será necesario alimentarlo con una fuente externa.

Leds

Se seleccionaron LEDs como elementos de salida para simular el funcionamiento de los actuadores del sistema, tales como ventiladores, iluminación artificial, extractores y sistemas de riego. Su uso permite representar de manera visual y sencilla las respuestas del sistema ante las diferentes condiciones evaluadas, facilitando la interpretación del comportamiento del prototipo durante la simulación.

Buzzer

El buzzer fue seleccionado como dispositivo de alerta sonora, su activación se asocia principalmente al uso del control manual, permitiendo generar una señal audible que advierte al usuario sobre cambios en el estado del sistema.

Diseño y conexión del circuito

La implementación del sistema se realizó en la plataforma de simulación Wokwi, lo que permitió validar su comportamiento sin necesidad de disponer de los componentes físicos.

En cuanto a la conexión de los dispositivos, el sensor DHT22 cuenta con cuatro pines: VCC, GND, DATA y NC. El pin NC no se utiliza, ya que no cumple una función eléctrica dentro del circuito. Para su funcionamiento, el pin VCC se conectó a 3.3 V del ESP32, el GND a tierra común y el pin de datos (DATA) al pin digital 21 del microcontrolador.

Por otro lado, el sensor de luminosidad tipo LDR, en su presentación como módulo, dispone de cuatro pines: VCC, GND, DO (salida digital) y AO (salida analógica). En este sistema se utilizó la salida analógica (AO), con el fin de obtener una lectura proporcional a la intensidad de luz. El pin VCC se conectó a 5 V del ESP32, el GND a tierra y el pin AO al pin 32 del microcontrolador. Es importante considerar que la alimentación a 5 V es posible cuando el ESP32 se encuentra energizado mediante USB; en otros casos, se requeriría una fuente externa adecuada.

El sensor ultrasónico HC-SR04 también posee cuatro pines: VCC, GND, TRIG y ECHO. Su funcionamiento se basa en la emisión (TRIG) y recepción (ECHO) de ondas ultrasónicas. En este caso, el pin TRIG se conectó al pin 25 y el pin ECHO al pin 33 del ESP32, además de su respectiva alimentación a 5 V y conexión a tierra común.

El sistema incluye un interruptor (switch) utilizado como control manual de emergencia. Este se conectó entre el pin 14 y GND, configurando el pin con resistencia interna pull-up y lógica de activación por flanco descendente (falling).

Para la generación de alertas sonoras, se incorporó un buzzer conectado al pin digital 2 y a GND, el cual se activa cuando se acciona el modo de emergencia.

En cuanto a las salidas visuales, se emplearon LEDs para simular los actuadores del sistema, todos conectados a pines digitales del ESP32 mediante resistencias de 220 Ω para limitar la corriente. La asignación de pines es la siguiente:

- LED amarillo (ventilación): pin 16
- LED rosado (humidificación): pin 4
- LED azul claro (nebulización): pin 19
- LED blanco (extractores): pin 18
- LED azul (iluminación): pin 17
- LED rojo (llenado de tanque): pin 5

El LED rojo se activa durante el proceso de llenado del tanque y se apaga cuando este alcanza el nivel máximo.

En la Figura 1 se presenta el esquema general del circuito, donde se pueden observar las conexiones realizadas entre los sensores, actuadores y el microcontrolador ESP32.

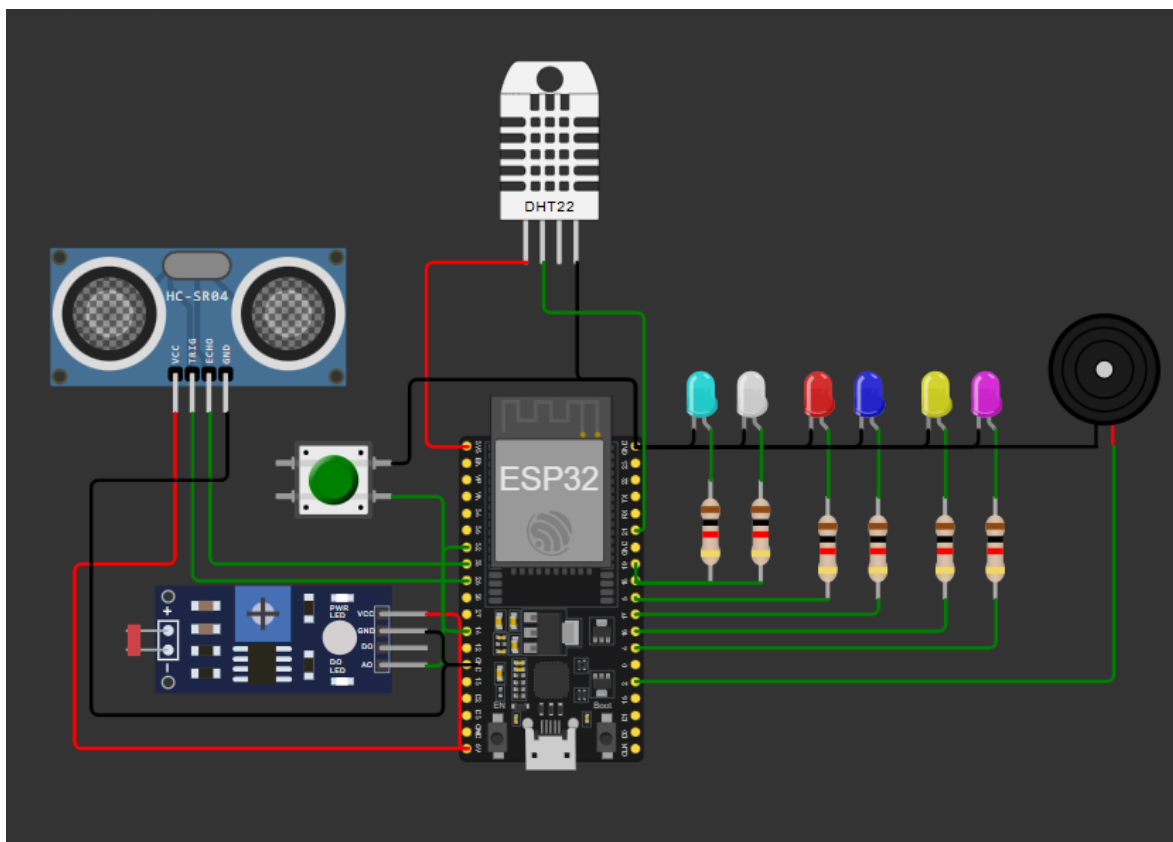


Figura 5. Conexiones del circuito

Con el fin de organizar y resumir la información de manera clara, se elaboró la Tabla 2, en la cual se especifica la asignación de pines de cada uno de los componentes utilizados en el sistema.

Tabla 2. Asignación de pines del ESP32

Componente	Pin ESP32	Tipo de Señal	Alimentación	Descripción
DHT22 (DATA)	21	Digital	3.3 V	Lectura de temperatura y humedad
LDR (AO)	32	Analógica	5 V	Lectura de luminosidad
HC-SR04 (TRIG)	25	Digital (out)	5 V	Envío de pulso ultrasónico

Componente	Pin ESP32	Tipo de Señal	Alimentación	Descripción
HC-SR04 (ECHO)	33	Digital (in)	5 V	Recepción del pulso
Switch	14	Digital	3.3 V (pull-up)	Control manual de emergencia
Buzzer	2	Digital	3.3 V	Alerta sonora
LED ventilación	16	Digital	3.3 V	Indicador de ventilación
LED humidificación	4	Digital	3.3 V	Indicador de humidificación
LED nebulización	19	Digital	3.3 V	Indicador de nebulización
LED extractores	18	Digital	3.3 V	Indicador de extracción
LED iluminación	17	Digital	3.3 V	Indicador de iluminación
LED nivel de tanque	5	Digital	3.3 V	Indicador de llenado

Lógica de Control del Sistema

El sistema de control fue desarrollado en lenguaje MicroPython, implementado sobre el microcontrolador ESP32. El programa se encarga de leer continuamente las variables ambientales mediante los sensores, procesar esta información y ejecutar acciones en función de las condiciones establecidas.

Inicialmente, se realiza la importación de librerías necesarias para el funcionamiento del sistema. Se utiliza la librería machine para la gestión de pines digitales y analógicos, generación de señales PWM y control de interrupciones. Asimismo, se emplea utime para el manejo de temporización mediante funciones como retardos y conteo de tiempo. Adicionalmente, se importa la librería math para operaciones matemáticas y dht para la comunicación con el sensor de temperatura y humedad.

Posteriormente, se inicializan variables que permiten el correcto funcionamiento del sistema. La variable inicio se utiliza para almacenar el tiempo inicial en el control del fotoperiodo, permitiendo gestionar los intervalos de encendido y apagado de la iluminación. Por su parte, la variable estado_led controla el estado del sistema de iluminación, diferenciando entre estados como encendido y apagado.

En cuanto a los sensores, se definen variables asociadas a los pines del sensor ultrasónico (TRIG y ECHO), las cuales permiten la medición de distancia para el control del nivel de agua. Para el sensor de luminosidad, se inicializa un objeto ADC asociado al pin

correspondiente, junto con constantes como GAMMA y RL10, necesarias para el cálculo de la intensidad lumínica en lux.

Respecto al sistema de control manual, se inicializa el switch con una configuración de resistencia interna pull-up, lo que permite detectar cambios de estado mediante interrupciones. Además, se definen variables booleanas como `button_pressed` y `emergencia`, las cuales permiten gestionar el estado del sistema frente a una activación manual.

Para la salida de alertas, se configura el buzzer mediante modulación por ancho de pulso (PWM), inicializándolo en estado apagado. Finalmente, se inicializa el sensor DHT22, el cual será utilizado para la lectura de temperatura y humedad, así como los pines correspondientes a los LEDs que representan los actuadores del sistema, tal como se logra observar en la figura 2.

```
from machine import Pin, ADC, time_pulse_us, PWM, disable_irq, enable_irq
from utime import sleep_ms, ticks_ms, ticks_diff
import math
import dht

inicio = 0
estado_led = "apagado"
#Ultra sonico
trig = Pin(25, Pin.OUT)
echo = Pin(33, Pin.IN)
led1=Pin(17,Pin.OUT)

#Fotoresistor
GAMMA = 0.7
RL10 = 50
adc = ADC(Pin(32))
led2=Pin(5,Pin.OUT)

#interrupcion
switch= Pin(14, Pin.IN, Pin.PULL_UP)
button_pressed = False
buzzer = PWM(Pin(2))
buzzer.duty(0)
emergencia = False
```

Figura 6. Definición de variables y librerías en el programa

En la Figura 3 se aprecia la primera función, correspondiente a la interrupción. Esta se activa únicamente cuando se presiona el botón, por lo cual su argumento es pin. Dentro de esta función se deshabilitan temporalmente las interrupciones para evitar conflictos mientras se ejecuta el código. Asimismo, dependiendo del cambio de estado del interruptor, se activa o desactiva el modo de emergencia. La instrucción:

“switch.irq(trigger=Pin.IRQ_FALLING, handler=fun_int)”, permite asociar la función de interrupción al pin del switch, definiendo que esta se ejecutará cuando ocurra un flanco descendente (*falling edge*), es decir, cuando el botón pase de estado alto a bajo.

```
#humedad y temperatura pines

dht_sensor = dht.DHT22(Pin(21))
led_ventilador= Pin(16, Pin.OUT)
led_humificador= Pin(4, Pin.OUT)
led_extractor= Pin(18, Pin.OUT)
led_nebulización = Pin(19, Pin.OUT)

def fun_int(pin):
    state = disable_irq()
    global emergencia
    global button_pressed
    button_pressed = True
    enable_irq(state)
    emergencia = not emergencia

switch.irq(trigger=Pin.IRQ_FALLING, handler=fun_int)
```

Figura 7. Definición variables humedad y temperatura, además de función de interrupción

A partir de este punto, en la programación se definen las funciones correspondientes a cada sistema de control, como se observa en la Figura 4.

En primer lugar, se define la función de alerta, la cual se activa cuando el estado de emergencia es verdadero (True). En este caso, se desactivan todos los mecanismos de

control (LEDs y actuadores) y se activa el buzzer como señal de advertencia. Mediante el método `duty()` se controla la intensidad o activación del sonido de la alerta.

Por otra parte, se tiene la función de lectura del sensor DHT22, la cual, a través de la librería `dht`, permite obtener los valores de temperatura y humedad. En caso de que no se logre realizar la lectura, la función retorna valores nulos (`None`).

Posteriormente, se define la función de control de temperatura, la cual, a partir del valor medido, lo imprime y toma decisiones mediante estructuras condicionales: si la temperatura es mayor a 37 °C se activa el ventilador, mientras que si es menor a 2 °C se activa el humidificador.

```
47 def alerta(emergencia):
48     if emergencia:
49         print("MODULO EMERGENCIA ACTIVADO")
50         led1.off()
51         led2.off()
52         led_ventilador.off()
53         led_humificador.off()
54         led_extractor.off()
55         led_nebulización.off()
56
57         buzzer.duty(512)
58         sleep_ms(200)
59         buzzer.duty(0)
60         sleep_ms(200)
61
62
63 def read_dht22():
64     try:
65         dht_sensor.measure()
66         temp = dht_sensor.temperature()
67         hum = dht_sensor.humidity()
68         return temp, hum
69     except OSError as e:
70         print("Error al leer el sensor: ", e)
71         return None, None
72
73 def condicion_temperatura(temp):
74     print(f"Temperatura: {temp} °C")
75     if temp > 37:
76         print("ventilador activado")
77         led_ventilador.on()
78         led_humificador.off()
79
80     elif temp < 2:
81         print("humidificador activado")
82         led_ventilador.off()
83         led_humificador.on()
84
85     else:
86         led_ventilador.off()
87         led_humificador.off()
88     print(f"-----")
89
```

Figura 8. Función de alerta, lectura DHT22 y control de temperatura

Como se ilustra en la Figura 5, se define la función de control de humedad, la cual toma decisiones a partir de la humedad relativa medida previamente. Se imprime el valor y, mediante condicionales, se determina que si la humedad es menor al 40% se activa el

sistema de nebulización; mientras que, si la humedad es mayor al 80% y la temperatura supera los 37 °C, se activan los extractores.

Adicionalmente, se encuentra la función de medición de distancia para el nivel de agua en el tanque de riego. En esta, se envía un pulso mediante el pin trig y se mide el tiempo de respuesta en el pin echo. A partir de la duración del pulso, se calcula la distancia multiplicando por 0.0343 (velocidad del sonido en cm/ μ s) y dividiendo entre 2, ya que el recorrido corresponde a ida y vuelta. Si no se obtiene una medición válida, la función retorna None.

```
89
90 def condicion_humedad(hum,temp):
91     print(f"Humedad: {hum} %")
92     if hum < 40:
93         print("nebulizacion activada")
94         led_nebulización.on()
95         led_extractor.off()
96
97     elif hum > 80 and temp > 37:
98         print("extractor activado")
99         led_extractor.on()
100        led_nebulización.off()
101    else:
102        led_nebulización.off()
103        led_extractor.off()
104
105 def medir_distancia():
106     trig.off()
107     sleep_ms(2)
108     trig.on()
109     sleep_ms(10)
110     trig.off()
111
112     # Medir duración del pulso en microsegundos
113     duracion = time_pulse_us(echo, 1, 30000) # timeout 30 ms
114     if duracion < 0:
115         return None
116
117     # Convertir a distancia en cm
118     distancia = (duracion * 0.0343) / 2
119     return distancia
```

Figura 9. Función de control de humedad y medición de distancia

En la Figura 6 se presenta la función de control del nivel del tanque, donde, a partir de la distancia medida, se imprime el valor y se toman decisiones según rangos establecidos. En este caso, se utiliza un LED indicador (LED2) para representar el estado del nivel del agua.

También se observa la función de cálculo de iluminación en lux, donde la señal analógica del sensor se convierte primero a voltaje, luego a resistencia y finalmente a una estimación en lux mediante una ecuación característica del fotoresistor.

```

122 def sensor_distancia(distancia):
123     if distancia is not None:
124         print(f"-----Mediciones-----")
125
126         print("Distancia:", round(distancia, 2), "cm")
127         if distancia < 180 and distancia > 20:
128             print("Llenando tanque...")
129
130             led2.on()
131         elif distancia > 180:
132             print("Tanque bajo, activando bomba de llenado...")
133
134             led2.on()
135         elif distancia < 20:
136             print("Tanque lleno, bomba de llenado cerrada")
137
138             led2.off()
139     else:
140         print("Error en la medición")
141         print(f"-----")
142 def lux():
143     analog_value = adc.read()
144     voltage_ldr = analog_value / 4095 * 5.0
145     resistance = (5 / voltage_ldr) - 1
146     resistance = 10000 / resistance
147     # Calcular la intensidad de luz en lux usando la fórmula proporcionada
148     lux = pow(RL10 * 1e3 * pow(10, GAMMA) / resistance, (1 / GAMMA))
149     return lux

```

Figura 10. Función control de distancia en llenado del tanque y cálculo de lux

Finalmente, en la Figura 7 se define la función de control del sistema de iluminación. En esta se utilizan variables globales (inicio y estado_led) para gestionar los tiempos de encendido y apagado. Se obtiene el tiempo actual mediante ticks_ms() y se imprime el valor de lux.

Si no hay lectura válida, el LED se apaga. Si el nivel de luz es menor a 100 lux, el sistema enciende el LED y registra el tiempo de inicio. Luego, mediante condiciones adicionales, se controla que el LED permanezca encendido durante un tiempo determinado (simulado

como 2000 ms) y posteriormente se apague durante un periodo de descanso (simulado como 10000 ms), representando un ciclo de iluminación para la planta.

```
150
151 def sensor_luz(lux):
152     global estado_led
153     global inicio
154     ahora = ticks_ms()
155     print("Lux:", round(lux, 2), "lux")
156     if lux is None:
157         print("Error: no se pudo leer el sensor de luz")
158         led1.off()
159         estado_led = "apagado"
160     # Se utiliza la variable estado_led para poder acceder al condicional que nos permite controlar el tiempo de encendido
161     elif lux < 100:
162         if estado_led == "apagado":
163             led1.on()
164             estado_led = "encendido"
165             inicio = ahora
166
167         elif estado_led == "encendido":
168             print("Luz encendida")
169             # Apagar después de 2 horas simuladas por los 2000 ticks
170             if ticks_diff(ahora, inicio) >= 2000:
171                 led1.off()
172                 estado_led = "apagado_espera"
173                 inicio = ahora
174
175         elif estado_led == "apagado_espera":
176             # Mantener apagado durante 10 horas simuladas por los 10000 ticks
177             if ticks_diff(ahora, inicio) >= 10000:
178                 led1.on()
179                 estado_led = "encendido"
180                 inicio = ahora
181
182     else:
183         led1.off()
184         estado_led = "apagado"
185         inicio = ahora
186     print(f"-----")
187
188
```

Figura 11. Función de control de iluminación

Por último, en la Figura 8 se presenta el ciclo principal while. En este se verifica si el sistema se encuentra en modo de emergencia; en caso afirmativo, se ejecuta la función de alerta continuamente.

Si no hay emergencia, se realizan las lecturas de los sensores (luz, temperatura, humedad y distancia). Posteriormente, se valida que todas las lecturas sean correctas y, en ese caso, se ejecutan las funciones de control correspondientes.

```
188
189 while True:
190     if emergencia:
191         alerta(emergencia)
192         continue
193     luxs = lux()
194     temperature, humidity = read_dht22()
195     distancia = medir_distancia()
196
197     if temperature is not None and humidity is not None and luxs is not None and distancia is not None:
198         sensor_distancia(distancia)
199         sensor_luz(luxs)
200         condicion_temperatura(temperature)
201         condicion_humedad(humidity,temperature)
202
203     else:
204         print("No se pudo leer algun sensor.")
205         if button_pressed == True:
206             button_pressed = False
207             alarm.on()
208         else:
209             alarm.off()
210
211     sleep_ms(1000)
212
213
```

Figura 12. Ciclo while

Validación del sistema

Con el fin de validar el correcto funcionamiento del sistema de control del invernadero, se realizaron pruebas experimentales bajo diferentes condiciones de operación, evaluando variables como temperatura, humedad, iluminación y nivel del tanque, junto con la respuesta de los actuadores.

Control de temperatura

Cuando la temperatura es menor a 2 °C, se activa el LED rosado, indicando condiciones de baja temperatura y la necesidad de humidificación.

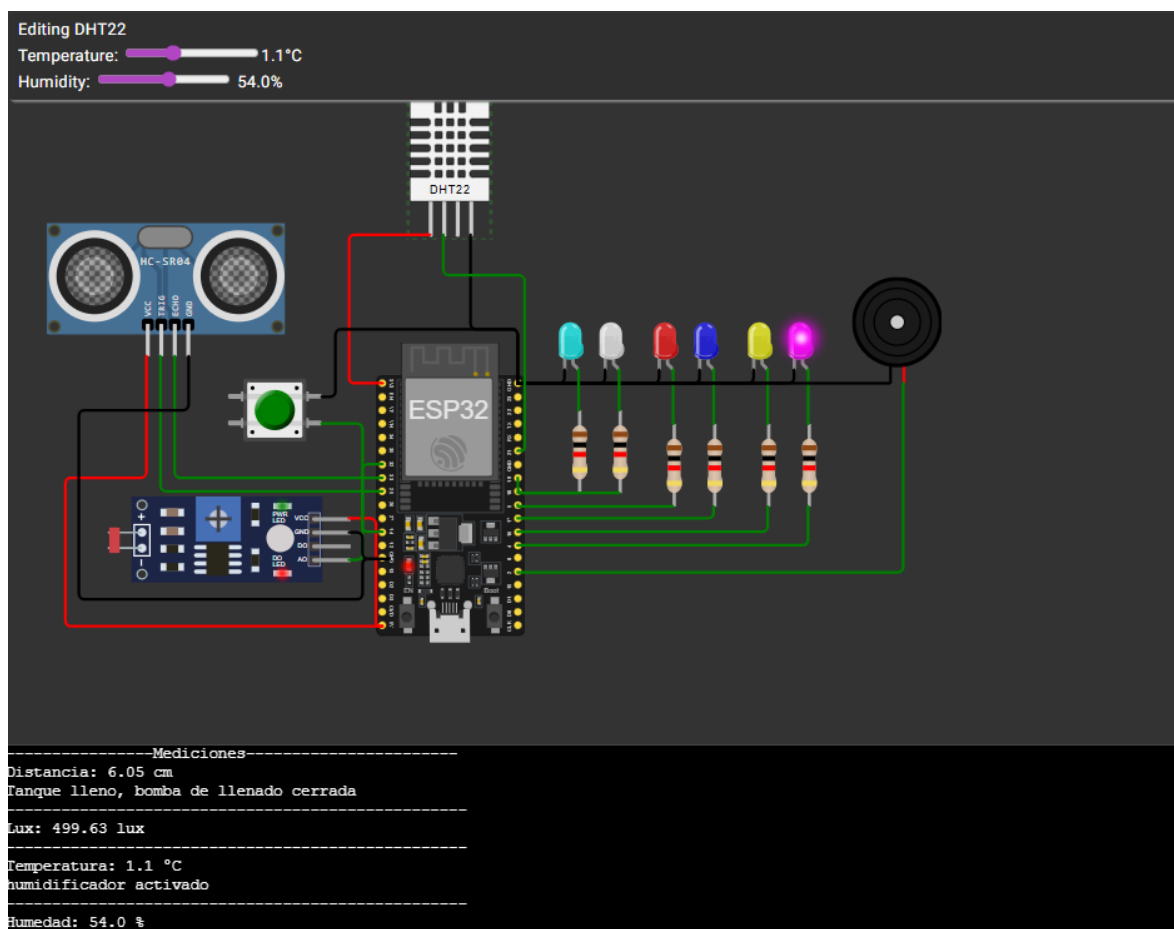


Figura 13. Activación del sistema ante temperatura baja.

Cuando la temperatura es mayor a 37 °C, se activa el LED amarillo, representando el encendido del sistema de ventilación para reducir la temperatura.

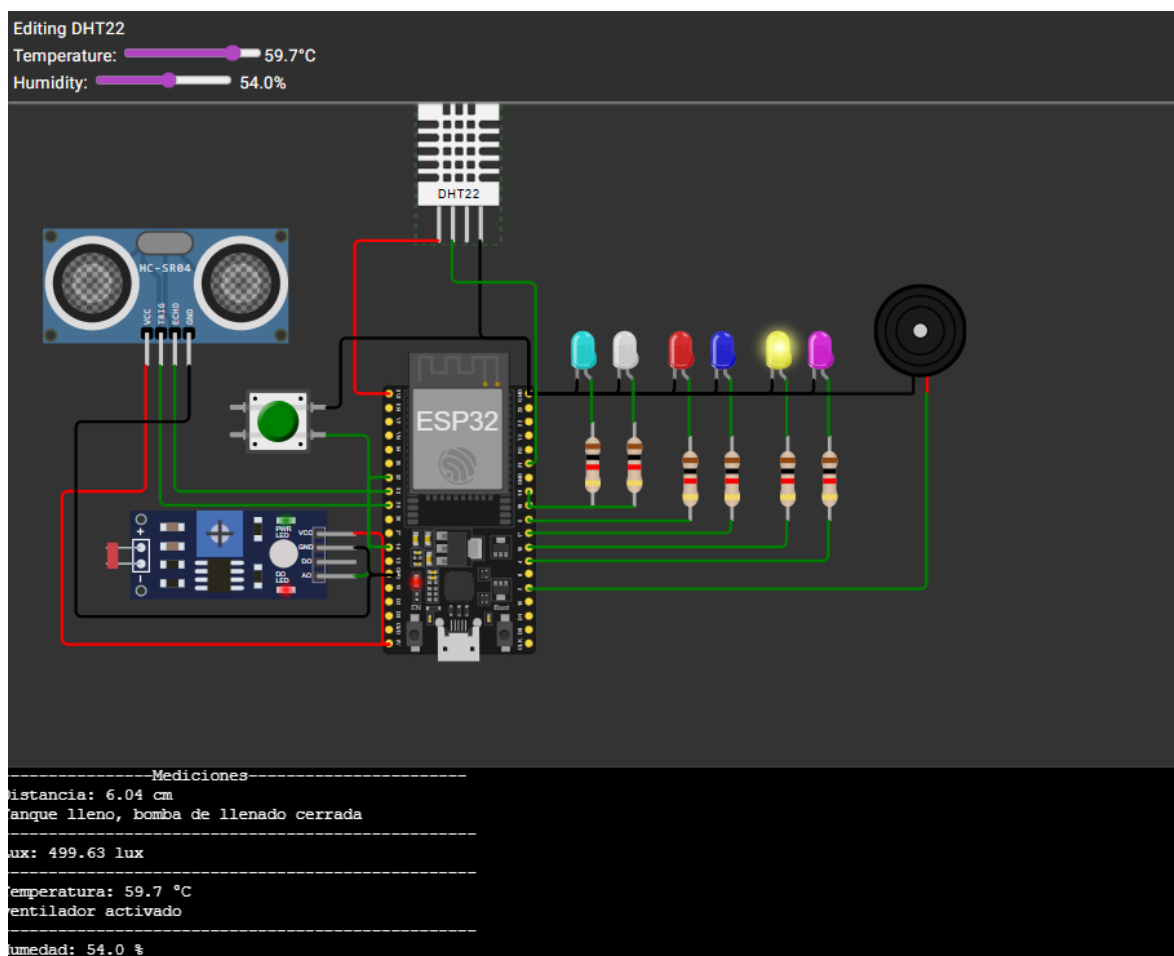


Ilustración 14. Activación del sistema de ventilación por alta temperatura.

Control de humedad

Cuando la humedad es mayor al 80% y la temperatura es mayor a 37 °C, se activan simultáneamente el LED blanco y el LED amarillo, indicando la operación del sistema de extracción para reducir tanto la humedad como la temperatura.

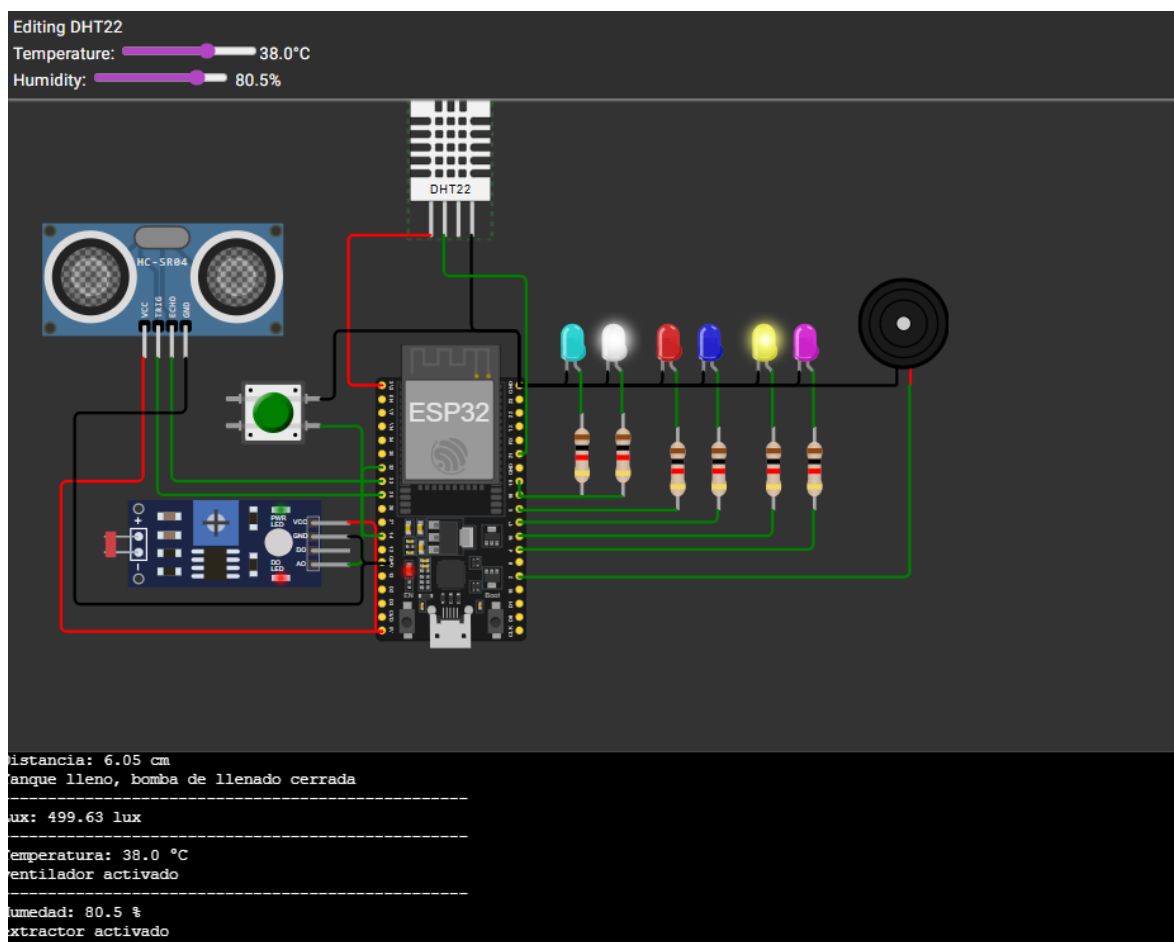


Figura 15. Activación del sistema de extracción por alta humedad y temperatura.

Cuando la humedad es menor al 40%, se activa el LED azul claro, representando el funcionamiento del sistema de nebulización para incrementar la humedad ambiental.

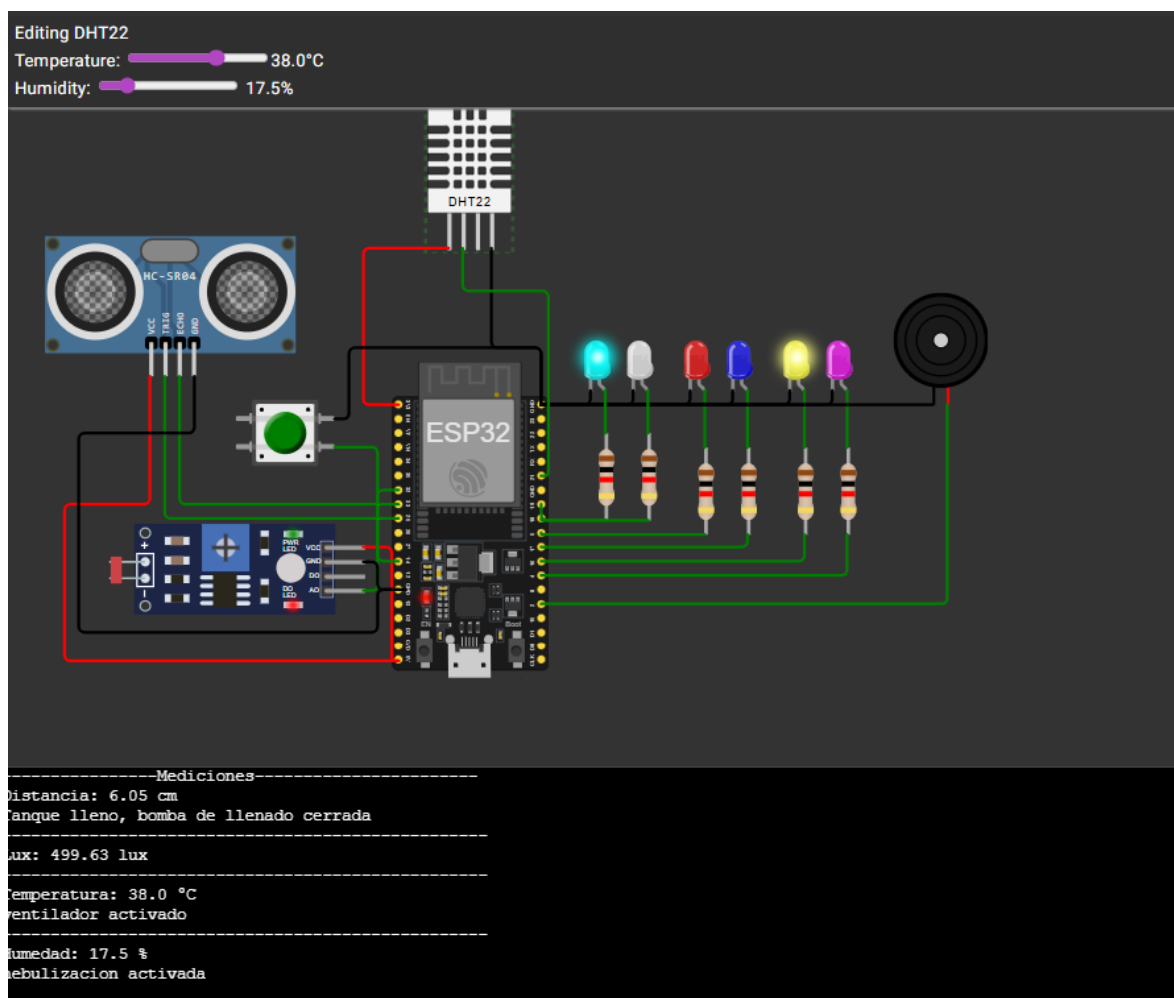


Figura 16. Activación del sistema de nebulización por baja humedad.

Control de nivel del tanque

Cuando la distancia medida se encuentra entre 20 cm y 180 cm, se enciende el LED rojo y se muestra el mensaje “Llenando tanque”, indicando que el sistema está en proceso de abastecimiento.

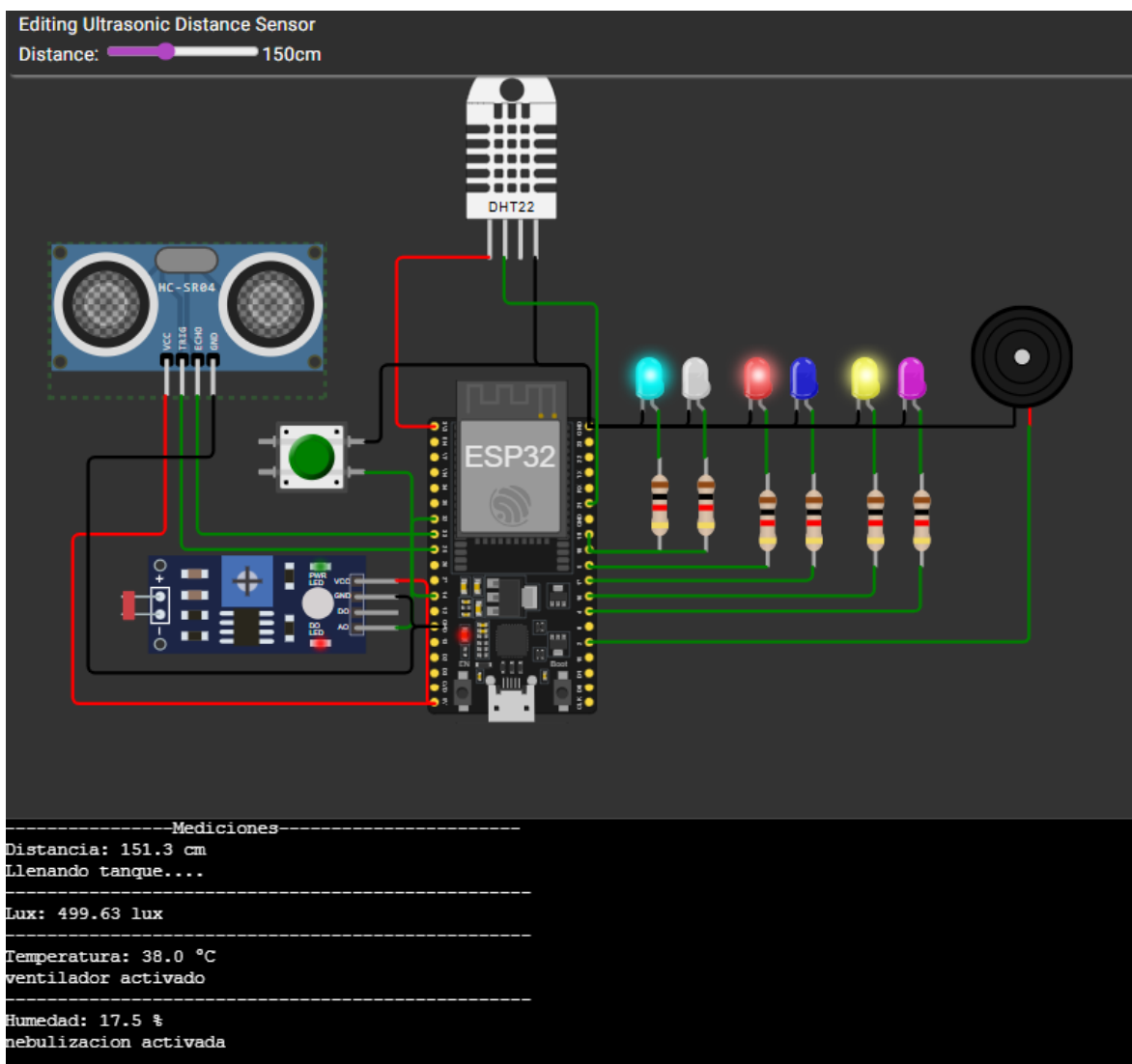


Figura 17. Estado de llenado del tanque de agua.

Cuando la distancia es mayor a 180 cm, el LED rojo permanece encendido y se muestra el mensaje “Tanque bajo, activando bomba de llenado”, indicando un nivel crítico de agua.

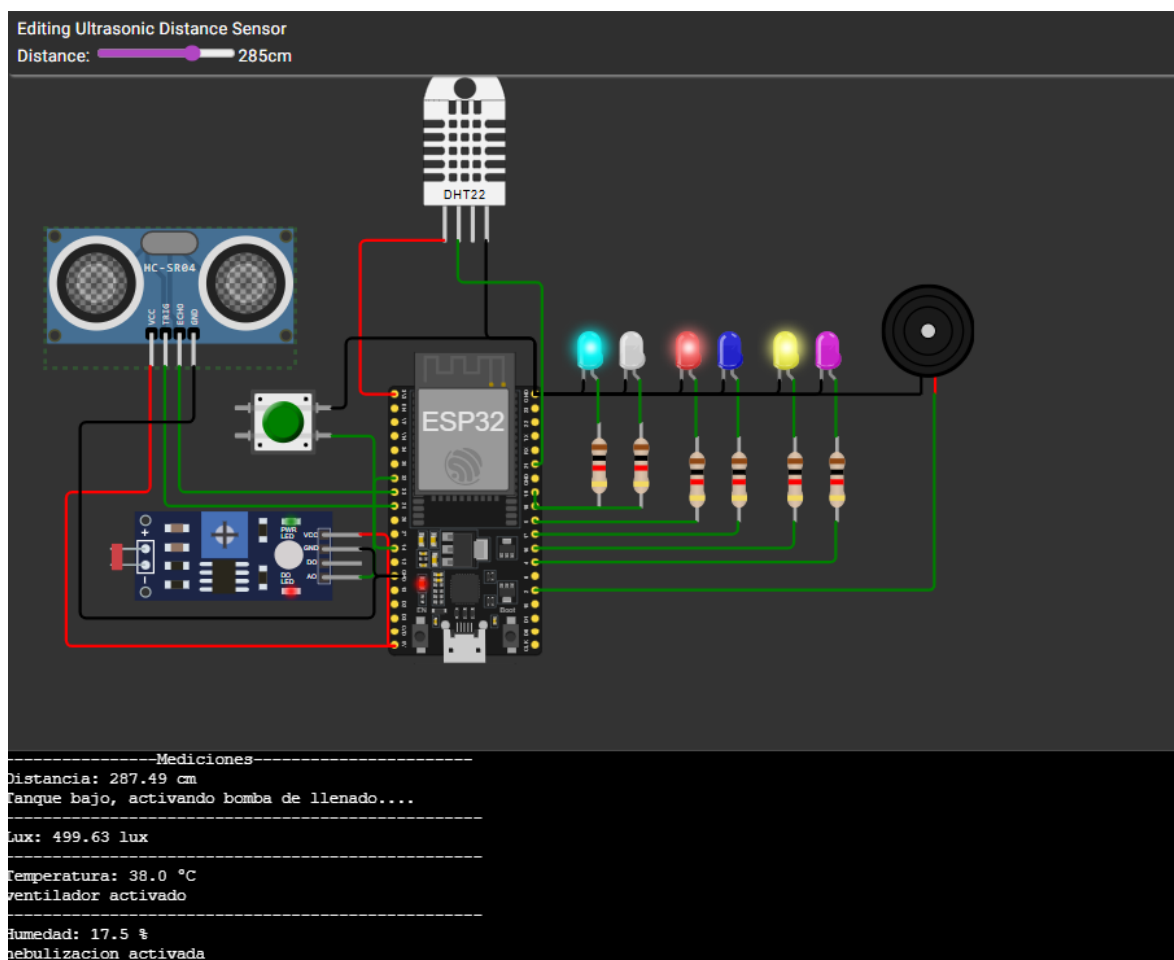


Figura 18. Detección de nivel bajo del tanque.

Cuando la distancia es menor a 20 cm, el LED rojo se apaga y se muestra el mensaje “Tanque lleno, bomba cerrada”, indicando que el tanque ha alcanzado su

capacidad máxima.

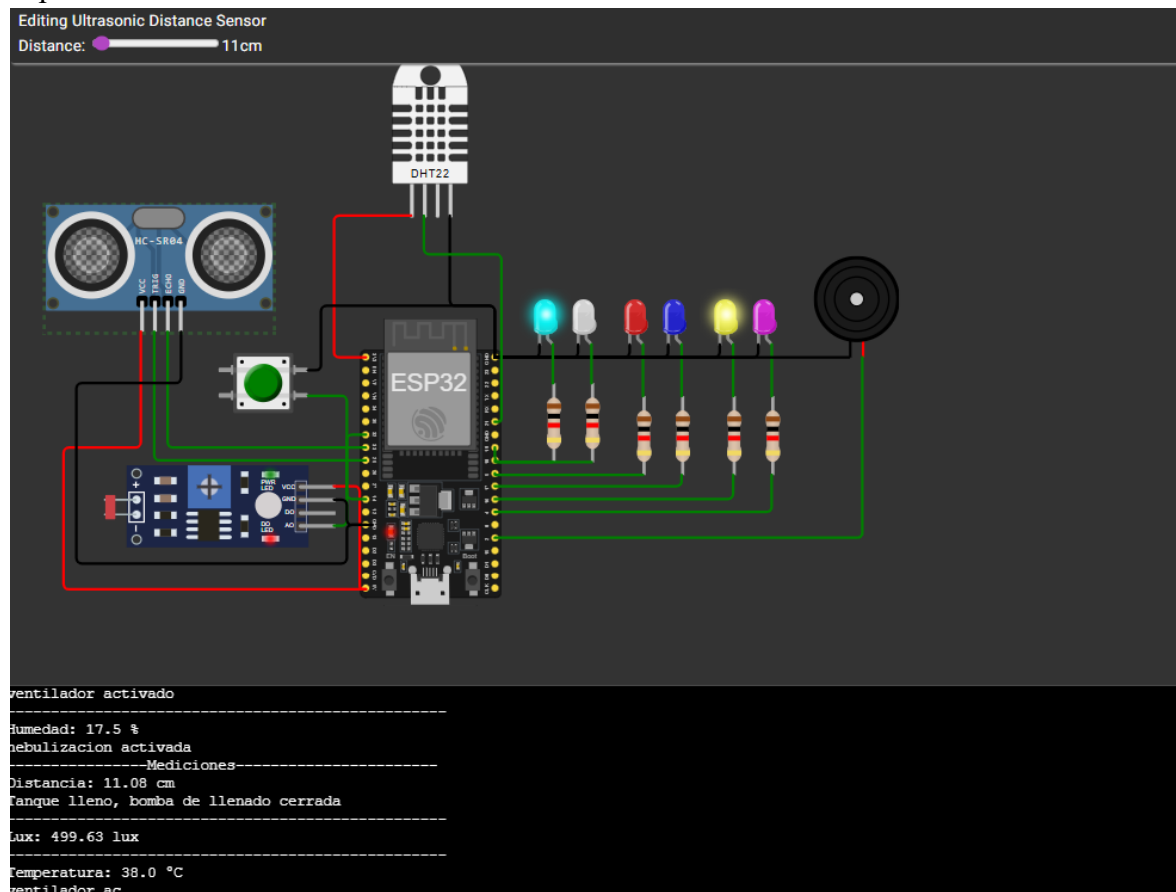


Figura 19. Detección de tanque lleno.

Control de iluminación

Cuando la iluminación es menor a 100 lux, se enciende la luz azul durante 2 segundos, mostrando el mensaje “Luz encendida”. Posteriormente, el sistema apaga la luz durante 10 segundos, simulando un ciclo de iluminación controlado.

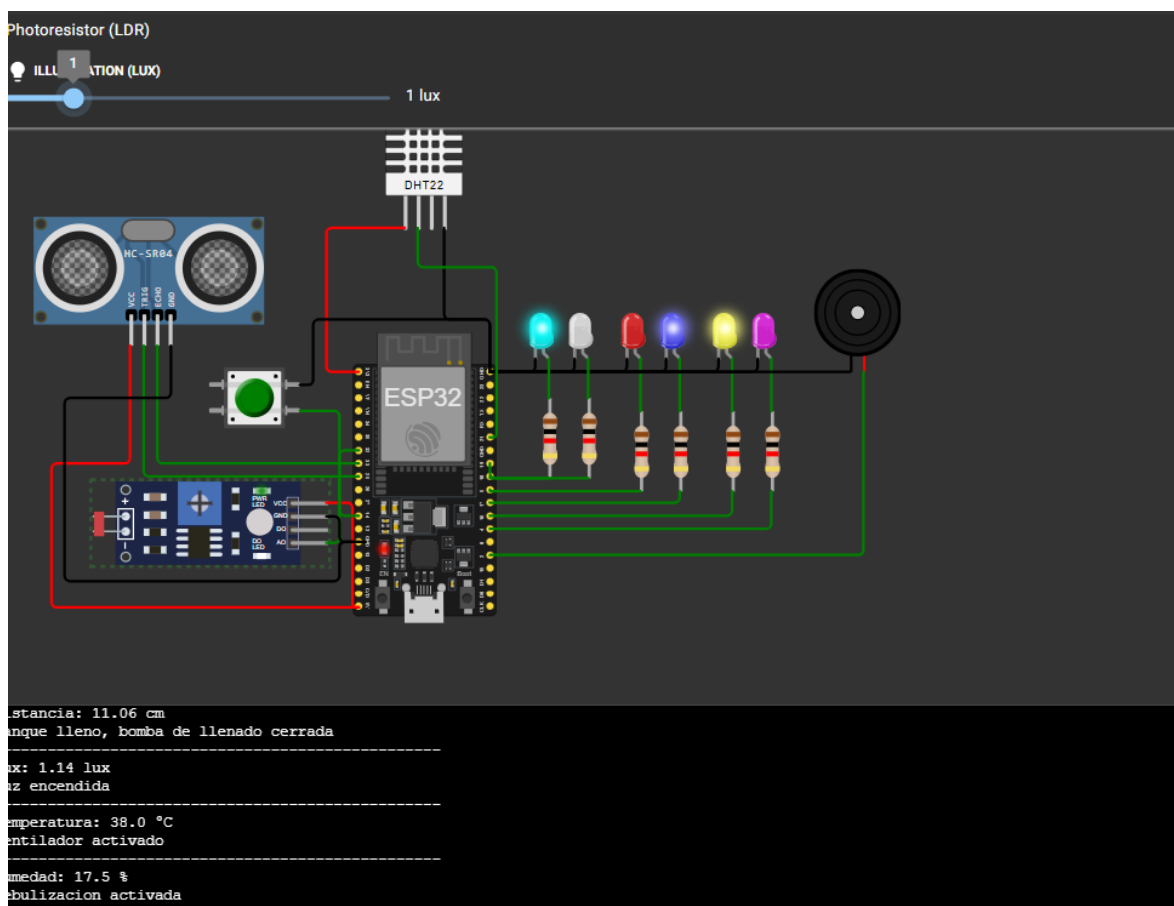


Figura 20. Ciclo de iluminación artificial por baja intensidad lumínica.

Modo de emergencia

Al activar el botón de emergencia, el sistema entra en modo de seguridad, donde:

- Se desactivan todos los actuadores y sistemas de control.
- Se activa el buzzer como señal de alerta.

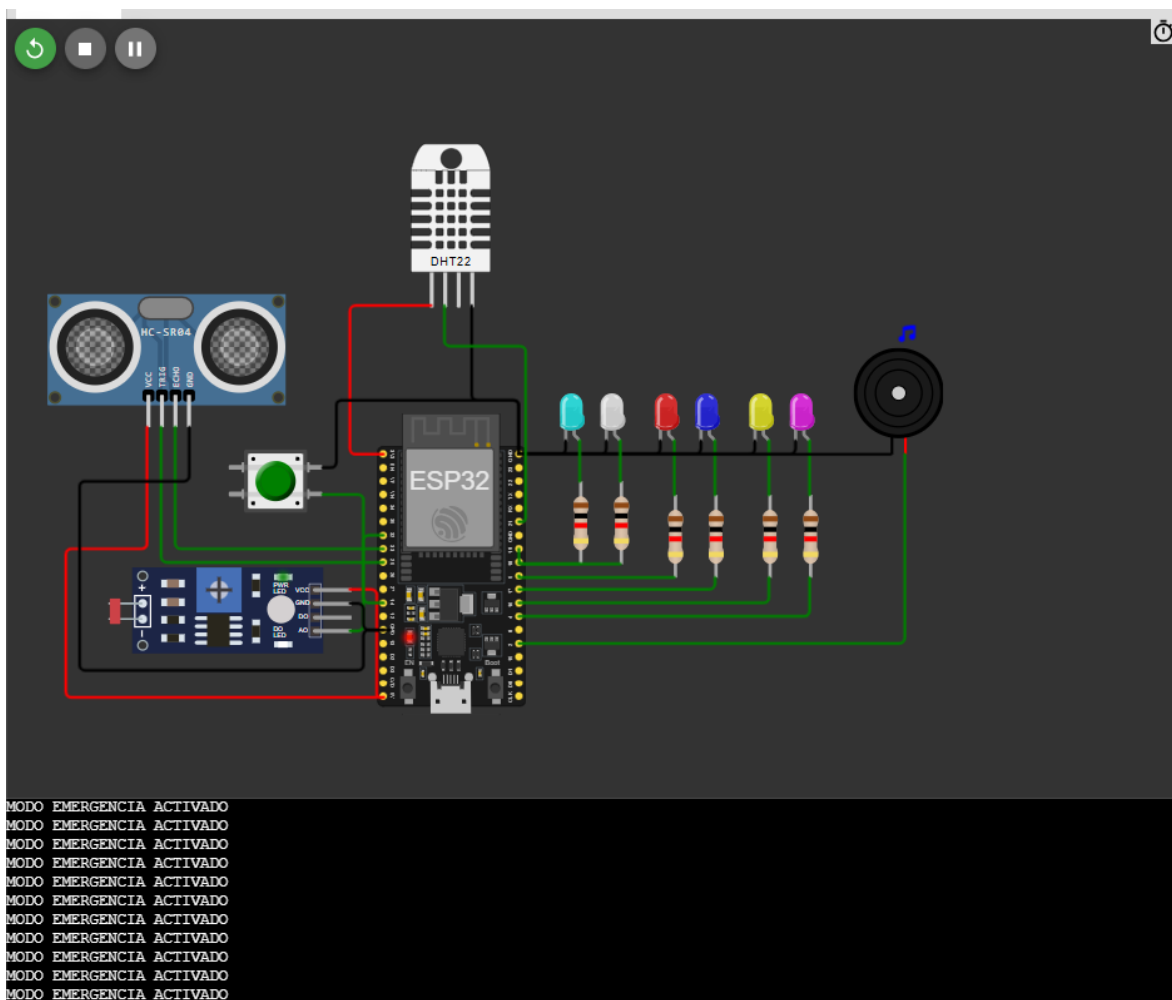


Figura 21. Activación del modo de emergencia del sistema

Resultados experimentales

En el primer caso, se registró una distancia de 11.06 cm, lo que indica que el tanque se encuentra lleno. Ante esta condición, el sistema respondió correctamente apagando la bomba de llenado. La iluminación registrada fue de 218.73 lux, valor superior al umbral establecido (100 lux), por lo cual no se activó el sistema de iluminación artificial. La temperatura fue de 6.2 °C, considerada baja, lo que activó el sistema de humidificación. Aunque la humedad fue alta (82.5%), no se activó el extractor debido a que la temperatura no superaba el valor crítico.

En el segundo caso, la distancia medida fue de 206.81 cm, indicando un nivel bajo del tanque. En respuesta, el sistema activó la bomba de llenado de manera adecuada. La

iluminación se mantuvo en 218.73 lux, por lo que no fue necesario activar la luz artificial. La temperatura alcanzó los 52 °C, superando el umbral establecido, lo que provocó la activación del sistema de ventilación. Adicionalmente, la humedad fue de 82.5% y, al combinarse con la alta temperatura, se activó el sistema de extracción, cumpliendo con la lógica de control programada.

Se observó que estas condiciones se repitieron en múltiples iteraciones del ciclo principal, lo cual demuestra que el sistema mantiene un comportamiento estable y consistente cuando las variables de entrada no cambian.

Los resultados obtenidos se resumen en la Tabla 3, donde se evidencia la correcta respuesta del sistema ante diferentes condiciones de operación.

Tabla 3. Mediciones simuladas

Caso	Distancia (cm)	Nivel del tanque	Acción tanque	Lux	Acción iluminación	Temperatura (°C)	Acción temperatura	Humedad (%)	Acción humedad
1	11.06	Tanque lleno	Bomba apagada	218.73	No activa luz	6.2	Humidificador activado	82.5	Sin acción
2	206.81	Tanque bajo	Bomba activada	218.73	No activa luz	52.0	Ventilador activado	82.5	Extractor activado
3	206.81	Tanque bajo	Bomba activada	218.73	No activa luz	52.0	Ventilador activado	82.5	Extractor activado
4	206.81	Tanque bajo	Bomba activada	218.73	No activa luz	52.0	Ventilador activado	82.5	Extractor activado

Conclusión

A partir del desarrollo de este proyecto, se pudo evidenciar la importancia de implementar sistemas de monitoreo en entornos como los invernaderos, donde el control de variables ambientales es clave para el crecimiento adecuado de las plantas. Más allá de la teoría, el trabajo permitió entender que factores como la temperatura, la humedad, la iluminación y el nivel de agua no solo deben medirse, sino también gestionarse de forma oportuna para evitar condiciones que puedan afectar el cultivo.

El uso del microcontrolador ESP32, junto con sensores y actuadores simulados, permitió construir un sistema capaz de responder automáticamente ante diferentes escenarios, lo que demuestra su utilidad en aplicaciones reales. Además, el desarrollo en MicroPython facilitó la implementación de la lógica de control, haciendo más claro el proceso de lectura de datos y toma de decisiones dentro del sistema.

Por otro lado, las pruebas realizadas en el entorno de simulación permitieron validar el funcionamiento del sistema, evidenciando que las respuestas generadas corresponden correctamente a las condiciones establecidas. Sin embargo, también se debe tener en cuenta que, en un entorno real, pueden presentarse variaciones o limitaciones que requieren ajustes adicionales, especialmente en la calibración de sensores y en la integración física de los componentes.

En general, este proyecto permitió no solo aplicar conceptos de sistemas embebidos, sino también comprender cómo la automatización puede aportar soluciones prácticas en el ámbito agrícola. Esto abre la posibilidad de seguir mejorando el sistema e implementar versiones más completas que integren nuevas funcionalidades y optimicen aún más el control del invernadero.

Referencias

- [1] IBM, "¿Qué es un microcontrolador?," *IBM Think Topics*, 2024. [En línea]. Disponible en: <https://www.ibm.com/mx-es/think/topics/microcontroller>.
- [2] Deepsea Developments, "ESP32 Chip Explained: Features and Advantages," *Deepsea Dev Blog*, 2023. [En línea]. Disponible en: <https://www.deepseadev.com/en/blog/esp32-chip-explained-and-advantages/>.
- [3] CircuitState, "DOIT ESP32 DevKit V1 WiFi Development Board Pinout Diagram and Reference," *CircuitState*, 2022. [En línea]. Disponible en: <https://www.circuitstate.com/pinouts/doit-esp32-devkit-v1-wifi-development-board-pinout-diagram-and-reference/>.
- [4] SD Industrial, "Sensores: qué son, para qué sirven y tipos," *SD Industrial Blog*, 2023. [En línea]. Disponible en: <https://sdindustrial.com.mx/blog/sensores/>.
- [5] Naylamp Mechatronics, "Tutorial sensor de temperatura y humedad DHT11 y DHT22," *Naylamp Mechatronics Blog*, 2024. [En línea]. Disponible en: https://naylampmechatronics.com/blog/40_tutorial-sensor-de-temperatura-y-humedad-dht11-y-dht22.html.
- [6] Mecatrónica LATAM, "Fotoresistencia LDR," *Mecatrónica LATAM*, 2024. [En línea]. Disponible en: <https://www.mecatronicalatam.com/es/tutoriales/sensores/sensor-de-luz/ldr/>.
- [7] Parsek, "Tecnología análoga: orígenes y aplicaciones," *Parsek Blog*, 2024. [En línea]. Disponible en: <https://parsek.com.co/blogs/tecnologia-analoga-origenes-componentes-aplicaciones>.
- [8] Unit Electronics, "Módulo Sensor Fotoresistor LDR," *Unit Electronics Product Page*, 2024. [En línea]. Disponible en: <https://uelectronics.com/producto/modulo-sensor-fotoresistor-ldr/>.
- [9] Naylamp Mechatronics, "Sensor Ultrasonido HC-SR04," *Naylamp Mechatronics Blog*, 2024. [En línea]. Disponible en: <https://naylampmechatronics.com/sensores-proximidad/10-sensor-ultrasonido-hc-sr04.html>.
- [10] Tameson, "¿Qué es un actuador? - Tipos y funciones," *Tameson Blog*, 2024. [En línea]. Disponible en: <https://tameson.es/pages/actuador>.

[11] Visual Led, "¿Qué es un LED? Definición y funcionamiento," *Visual Led Blog*, 2024. [En línea]. Disponible en: <https://visualled.com/glosario/que-es-un-led/>.

[12] MicroPython, "MicroPython - Python for microcontrollers," *MicroPython Official Website*, 2024. [En línea]. Disponible en: <https://micropython.org/>.